

■ Stroke Prediction

Machine Learning Project

Healthcare Dataset — Binary Classification

Marwan Gharib & Mohamed Abdelrehim | NTI Program

5,110	12	4.9%	7
Records	Features	Stroke Rate	Models Tested

Table of Contents

Section	Page
1. Problem Statement & Objectives	2
2. Dataset Overview	2
3. Exploratory Data Analysis (EDA)	3
4. Data Preprocessing	3
5. Models & Results	4
6. OLS Normal Equation	5

7. LOG LOSS — Logistic Regression	5
8. Model Comparison Table	6
9. Feature Importance	6
10. Confusion Matrix — Visual	7
11. Grid Search CV	7
12. Randomized Search CV	8
13. Prediction Function	8
14. Conclusions	9

1. Problem Statement & Objectives

Stroke is the second leading cause of death globally, responsible for approximately 11% of total deaths. Early prediction can significantly improve patient outcomes by enabling timely intervention. This project builds a machine learning pipeline to predict stroke risk from patient health records, comparing multiple classification algorithms and identifying the most predictive features.

- Perform comprehensive EDA on healthcare stroke data
- Handle class imbalance using SMOTE
- Train and compare 7 ML models
- Compute OLS Normal Equation manually ($w = (X^T X)^{-1} X^T y$)
- Calculate LOG LOSS manually using to_categorical
- Build a real-time stroke risk prediction function

2. Dataset Overview

Property	Value
Source	Kaggle — Healthcare Dataset Stroke Data
Total Records	5,110 rows
Features	12 columns (10 features + id + target)
Target Variable	stroke (0 = No Stroke, 1 = Stroke)
Class Balance	4,861 No Stroke (95.1%) 249 Stroke (4.9%)
Missing Values	bmi: 201 missing → filled with Median

Feature Descriptions

Feature	Type	Description
id	int	Unique identifier (dropped)
gender	object	Male / Female
age	float	Patient age
hypertension	int	0 = No, 1 = Yes
heart_disease	int	0 = No, 1 = Yes
ever_married	object	Yes / No
work_type	object	Private / Self-employed / Govt / Children

Residence_type	object	Urban / Rural
avg_glucose_level	float	Average blood glucose level
bmi	float	Body Mass Index
smoking_status	object	formerly smoked / never smoked / smokes
stroke	int	TARGET (0 = No, 1 = Yes)

3. Exploratory Data Analysis (EDA)

Age Distribution	Stroke patients tend to be significantly older. The distribution shows a clear peak for stroke cases in the 60–80 age range.
Glucose vs BMI	Higher glucose levels (>150) appear more frequently among stroke patients. BMI alone shows weaker discrimination.
Hypertension	Patients with hypertension show a higher proportion of strokes compared to non-hypertensive patients.
Heart Disease	Presence of heart disease is associated with a higher stroke risk.
Correlation Heatmap	Age shows the strongest correlation with stroke (0.25), followed by hypertension (0.13) and heart_disease (0.13).
Class Imbalance	Only 4.9% of patients had a stroke — severe imbalance requiring SMOTE oversampling before training.

4. Data Preprocessing

Step 1 — Drop ID	Removed the `id` column — unique identifier with no predictive value.
Step 2 — Handle NaN	201 missing `bmi` values filled with the column median (28.1) to preserve all records.
Step 3 — Remove Outlier	1 row with gender = 'Other' removed (statistical noise).
Step 4 — Label Encoding	5 categorical columns encoded: gender, ever_married, work_type, Residence_type, smoking_status.
Step 5 — Train/Test Split	80% Train (4,087 rows) 20% Test (1,022 rows) with stratify=y to preserve class ratio.
Step 6 — StandardScaler	fit_transform on Train only → transform on Test. Brings all features to mean=0, std=1.
Step 7 — SMOTE	Before: 3,888 No Stroke vs 199 Stroke. After: 3,888 vs 3,888 — perfectly balanced training set.

■ ■ Key Note on SMOTE: SMOTE (Synthetic Minority Over-sampling Technique) is applied ONLY on the training set after scaling — never on the test set. This prevents data leakage and ensures honest evaluation.

5. Models & Individual Results

Model	Accuracy	F1-Score	AUC-ROC	Notes
Linear Regression	72.60%	22.65%	N/A	■ Not suitable — predictions go outside [0,1]
Logistic Regression	74.56%	23.53%	83.68%	■ Best AUC — ideal for binary classification
SVM (RBF Kernel)	75.44%	19.29%	76.42%	■ Good accuracy, decent AUC
Decision Tree	74.17%	22.81%	75.91%	■ Interpretable but lower AUC
Random Forest	93.05%	18.39%	77.15%	■ Highest accuracy, used for final prediction
KNN (k=5)	79.94%	14.94%	66.36%	■ Low F1 and AUC on stroke class
Gradient Boosting	87.96%	28.07%	80.37%	■ Best F1-Score, second best AUC

Logistic Regression — Detailed Report

Class	Precision	Recall	F1-Score	Support
No Stroke	0.99	0.74	0.85	972
Stroke	0.14	0.80	0.24	50
Accuracy			0.75	1022
Macro Avg	0.56	0.77	0.54	1022
Weighted Avg	0.94	0.75	0.82	1022

Confusion Matrix (Logistic Regression): $\begin{bmatrix} 722 & 250 \\ 10 & 40 \end{bmatrix}$ → Correctly identified 40 out of 50 stroke cases (Recall = 80%)

6. OLS Normal Equation — Manual Weight Calculation

The Ordinary Least Squares closed-form solution computes model weights directly without gradient descent:

$$w = (X^T X)^{-1} X^T y$$

```
intercept_col = np.ones((X_train_res.shape[0], 1)) X1 =  
np.concatenate([intercept_col, X_train_res], axis=1) z = np.linalg.inv(np.dot(X1.T,  
X1)) w = np.dot(z, np.dot(X1.T, y_train_res))
```

intercept	0.3232
coef1	0.0001
coef2	0.3240
coef3	0.0278

These weights match the sklearn LinearRegression coefficients — proving that sklearn internally uses the OLS solution. ■

7. LOG LOSS — Logistic Regression

Cross-entropy loss measures how well the predicted probabilities match the actual labels. Lower is better.

$$\text{Loss} = -1/N \times \sum y_i \cdot \log(p_i)$$

```
# sklearn LOG_LOSS y_pred_prob = log.predict_proba(X_test_scaled) LOG_LOSS =  
log_loss(y_test, y_pred_prob) # Manual LOG_LOSS from keras.utils import  
to_categorical y_test_cat = to_categorical(y_test) loss = -np.sum(y_test_cat *  
np.log(y_pred_prob), axis=1).mean() print("sklearn LOG_LOSS:", LOG_LOSS) # 0.5012  
print("Manual LOG_LOSS:", loss) # 0.5012
```

sklearn LOG_LOSS	Manual LOG_LOSS	Match?
0.5012	0.5012	■ Identical

8. Final Model Comparison Table

Model	Accuracy	F1-Score	AUC-ROC	Rank
Logistic Regression	74.56%	23.53%	83.68%	■ Best AUC
Gradient Boosting	87.96%	28.07%	80.37%	■ Best F1
Random Forest	93.05%	18.39%	77.15%	■ Best Acc
SVM (RBF)	75.44%	19.29%	76.42%	4th
Decision Tree	74.17%	22.81%	75.91%	5th
KNN (k=5)	79.94%	14.94%	66.36%	6th
Linear Regression	72.60%	22.65%	N/A	■ Not ideal

Best overall model for stroke prediction: Logistic Regression (AUC = 83.68%) → highest ability to distinguish stroke from non-stroke cases. Random Forest (Accuracy = 93.05%) is used in the prediction function due to its robustness across all features.

9. Feature Importance — Random Forest

Feature	Importance	Impact
age	0.3811	■ Very High — most predictive feature
smoking_status	0.1236	■ High
bmi	0.1232	■ High
avg_glucose_level	0.1204	■ High
work_type	0.0972	■ Medium
ever_married	0.0593	■ Medium
Residence_type	0.0271	■ Low
gender	0.0267	■ Low
hypertension	0.0252	■ Low
heart_disease	0.0162	■ Low

Key Insight: Age (38.1%) is by far the most important predictor of stroke. This aligns with medical literature — stroke risk increases significantly after age 55. Lifestyle factors (smoking, BMI, glucose) collectively account for ~37% of the model's predictive power.

10. Confusion Matrix — Visual

Confusion Matrix 4 classes:

	Actual No Stroke	Actual Stroke
Predicted No Stroke	722 TN	10 FN
Predicted Stroke	250 FP	40 TP

Logistic Regression — Confusion Matrix

Predicted →	No Stroke	Stroke
Actual: No Stroke	722 ■ TN	250 ■ FP
Actual: Stroke	10 ■ FN	40 ■ TP

False Negative = 10 — 10 / 50 = 0.2 = 20%. Medical Data False Negative —

11. Grid Search CV — Hyperparameter Tuning

Grid Search combinations parameters AUC-ROC.

```
from sklearn.model_selection import GridSearchCV
param_grid = { 'n_estimators': [50, 100, 200], 'max_depth': [3, 5, 10], 'min_samples_split': [2, 5] }
grid_search = GridSearchCV( RandomForestClassifier(random_state=42), param_grid, cv=5, # 5-Fold Cross Validation
scoring='roc_auc', n_jobs=-1 # CPU cores )
grid_search.fit(X_train_res, y_train_res)
print('Best Parameters:', grid_search.best_params_)
print('Best AUC-ROC :', grid_search.best_score_)
```

Parameter	Values Tested	Best Value
n_estimators	[50, 100, 200]	100
max_depth	[3, 5, 10]	10

min_samples_split	[2, 5]	2
-------------------	--------	---

12. Randomized Search CV — Hyperparameter Tuning

Randomized Search  combinations .  Grid Search —  parameters .

```
from sklearn.model_selection import RandomizedSearchCV param_dist = {
'n_estimators': [50, 100, 150, 200, 300], 'max_depth': [3, 5, 10, 15, None],
'min_samples_split': [2, 5, 10], 'min_samples_leaf': [1, 2, 4], } random_search =
RandomizedSearchCV( RandomForestClassifier(random_state=42), param_dist, n_iter=20,
#  20 combination  cv=5, scoring='roc_auc', random_state=42 )
random_search.fit(X_train_res, y_train_res) print('Best Parameters:',
random_search.best_params_) print('Best AUC-ROC :', random_search.best_score_)
```

Grid Search vs Randomized Search

	Grid Search	Randomized Search
	 combinations	20 combination 
		
		
 params	 (2-3)	 (4+)
	Final Tuning	Exploration 

 Best Practice:  Randomized Search  parameters  Grid Search    Random Forest.

13. Prediction Function

A ready-to-use function that takes patient data and returns real-time stroke risk prediction using the trained Random Forest model:

```
def predict_stroke(age, hypertension, heart_disease, avg_glucose_level, bmi, ...):
patient = np.array([[gender, age, hypertension, heart_disease, ever_married,
work_type, Residence_type, avg_glucose_level, bmi, smoking_status]]) patient_scaled
= scaler.transform(patient) pred = rf.predict(patient_scaled)[0] prob =
rf.predict_proba(patient_scaled)[0][1] print(f"Prediction : STROKE RISK or No
Stroke") print(f"Probability : {prob*100:.1f}%")
```

Patient Profile	Prediction	Probability
High Risk (Age=75, Hypertension=1, Heart Disease=1, Glucose=220, BMI=33)	STROKE RISK 	86.0%

Medium Risk (Age=55, Hypertension=1, Heart Disease=0, Glucose=150, BMI=27)	Stroke ■	12.0%
Low Risk (Age=25, Hypertension=0, Heart Disease=0, Glucose=88, BMI=21)	No Stroke ■	0.0%

14. Conclusions

■ Best AUC Model	Logistic Regression (83.68%) — best for detecting stroke risk
■ Best Accuracy	Random Forest (93.05%) — used in the final prediction function
■ Best F1 for Stroke	Gradient Boosting (28.07%) — best at identifying actual stroke cases
■ Key Risk Factor	Age is the #1 predictor (38.1% importance)
■ SMOTE Impact	Balanced training set from 3,888:199 to 3,888:3,888
■ OLS Validation	Manual $w = (X^T X)^{-1} X^T y$ matches sklearn coefficients exactly
■ LOG LOSS Validation	Manual to_categorical loss = sklearn log_loss = 0.5012
■ Confusion Matrix	False Negative = 10 only — ■■■■■■■■ ■■■■ 10 ■■■■■ ■■■■ ■■ ■■ ■■■ 50
■ Grid Search	Best params: n_estimators=100, max_depth=10, min_samples_split=2
■ Randomized Search	Faster exploration of 20 random combinations from wider param space
■ Challenge	Severe class imbalance (4.9% stroke) — SMOTE helps but F1 on stroke class remains low